TE MINI PROJECT LOGBOOK

AUTHBLOCK

An Enterprise-Grade Decentralized Application for Academic Credential Verification Using Blockchain

GROUP MEMBERS

1. Jason Gonsalves (Roll No. 10246)
2. Ashley Almeida (Roll No. 10227)

GUIDE/SUPERVISOR

Prof. Prajakta Dhamanskar

Department of Computer Engineering
Fr. Conceicao Rodrigues College of Engineering, Bandra
University of Mumbai
(AY 2025-2026)

DEPARTMENT OF COMPUTER ENGINEERING
Fr. Conceicao Rodrigues College of Engineering
Bandra (W), Mumbai - 400050
University of Mumbai

Student Details

Semester: VI (Sixth Semester)
Subject Code: 25PCC13CE18

Project Title: AUTHBLOCK — An Enterprise-Grade Decentralized Application for Academic Credential Verification Using Blockchain

Category of Project: Application / Product

Team Members:

Roll Number	Name
10246	Jason Gonsalves
10227	Ashley Almeida

Project Guide: Prof. Prajakta Dhamanskar, Department of Computer Engineering, C.R.C.E., Bandra

Course Outcomes:

Semester VI

	Mini Project 25PCC13CE18 TE COMPS A & B
CO1	Identify and analyze problems related to society, research, innovation, and entrepreneurship through a comprehensive literature survey.
CO2	Formulate and apply appropriate methodologies using engineering knowledge and skills to develop effective solutions.
CO3	Validate, verify, and evaluate the impact of solutions using test cases, benchmark data, theoretical inferences, experiments, or simulations.
CO4	Adopt standard engineering practices and project management principles while ensuring sustainability and ethical considerations.
CO5	Develop technical competency and lifelong learning through self-directed learning, participation in competitions, hackathons, and exposure to industry trends.
CO6	Enhance communication and teamwork skills through technical report writing, presentations, and collaborative group work.

Programme Outcomes

Engineering Graduates will be able to

- **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **Design/Development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and the cultural, societal, and environmental considerations.
- **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis, and interpretation of data, and synthesis of the information to provide valid conclusions.
- **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling of complex engineering activities with an understanding of the limitations.
- **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and the need for sustainable development.

- **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **Project Management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **Life-long learning:** Recognized the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Programme Specific Outcomes

The student will have the ability to

- Develop Artificial Intelligence and Machine Learning systems.
- Apply cyber security mechanisms to ensure the protection of information technology assets.

CO-PO-PSO Mapping with justification:

CO	CO Statements	PO	Level	Justification
CO1	Identify societal/research/innovation/entrepreneurship problems through appropriate literature surveys	PO1 <1.3.1 1.4.1> PO6 <6.1.1> PO7 <7.1.1> PO8 <8.2.2>	2 2 1 1	Students apply engineering fundamentals and computer science principles to find real-world credential fraud problems. They identify existing solutions (Blockcerts, Hyperledger) and compare them. They identify risks/impacts wrt environment and sustainability. They apply moral & ethical principles (IAM Least Privilege, key management).
CO2	Identify methodology for solving above problem and apply engineering knowledge and skills to solve it	PO2 <2.1.1 2.1.2>	2 1 3 1	Students define problem statements (academic credential fraud), objectives and scope. They identify modules

		PO3 <3.1.1> PO4 <4.1.2 4.3.2> PO5 <5.3.1> PO8 <8.2.2>	1	(Smart Contract, PDF Gen, S3, SNS/SQS/Lambda), datasets (student CSV) and computing resources (Alchemy RPC, NeonDB, AWS). They define algorithms (SHA-256, storeHash, verifyHash), test cases for proposed solution, and create documentation.
CO3	Validate, Verify the results using test cases/benchmark data/theoretical inferences/experiments/simulations	PO3 <3.4.3>	1	Students verify hash verification accuracy (100% on 50 docs), tamper-detection rate (100% on 20 modified PDFs), email delivery rate (100%), and IAM policy boundary validation through controlled tests.
CO5	Use standard norms of engineering practices and project management principles during project work	PO10 <10.1.1, 10.1.2, 10.1.3>	2	Students produce well-constructed engineering documents (mini project report). They use Git branching workflows, sprint-based project management, and show logical progression of ideas through architecture diagrams, UML, and weekly logbook entries.
CO8	Demonstrate capabilities of self-learning, leading to lifelong learning.	PO12 <12.1.2>	1	Students independently learned Next.js 14 App Router, Solidity 0.8.x, AWS SDK v3 (SNS/SQS/Lambda/SES), Hardhat deployment, and Firebase MFA – tools not covered in classroom curriculum.
CO9	Develop interpersonal skills to work as a member of a group or as leader.	PO8 <8.2.2>	1	Team of 2 students applied ethical principles

		PO9 PO10 <10.2.1, 10.2.2>	3 2	in credential system design. Demonstrated collaborative division of responsibilities (frontend/backend split), regular guide meetings, peer code reviews, and prepared joint project presentations and report.

WEEK – 01
Date: From 01 Jan 2026 To: 07 Jan 2026

Progress Achieved:
Kickoff meeting with Prof. Prajakta Dhamanskar to finalize project scope for Semester VI. Reviewed Semester V deliverables: problem statement, literature survey, and system architecture.

Confirmed technology stack: Next.js 14 (App Router), NeonDB Serverless PostgreSQL, Firebase Auth with MFA, AWS (S3, SNS, SQS, Lambda, SES, CloudWatch, CloudTrail), Solidity 0.8.x on Ethereum Sepolia Testnet, Alchemy/Infura RPC.

Set up GitHub repository with branch protection rules (main, dev, feature/* strategy). Configured GitHub Projects Kanban board for sprint tracking. Initialized Next.js 14 project with TypeScript and Tailwind CSS using create-next-app.

Created AWS IAM user (authblock-service) with inline Least Privilege policy scoped to specific S3 bucket ARN, SNS topic ARN, and SES sending identity. No wildcard permissions granted.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Initialized Next.js 14 project with TypeScript, Tailwind CSS. Configured .env.local template with all required variable placeholders. Set up GitHub repository and branch protection rules.	Researched Next.js 14 App Router architecture and serverless PostgreSQL (NeonDB) patterns. Created AWS IAM user with Least Privilege inline policy. Documented all required environment variables.	N/A	N/A

Mentor's Suggestions:

Ensure all API keys and private keys are never committed to version control. Use .gitignore strictly. Document the architecture before writing a single line of code. Review Solidity security best practices (reentrancy, access control) before contract development.

Signature:
Team Member 1: Jason Gonsalves
Team Member 2: Ashley Almeida

Project guide:
Signature: _____
Date: _____

WEEK – 02
Date: From 08 Jan 2026 To: 14 Jan 2026

Progress Achieved:
Completed detailed system architecture design: four-layer pipeline — Presentation (Next.js frontend), Application (API routes/serverless functions), Data (NeonDB + Amazon S3), and Blockchain (Ethereum Sepolia via Alchemy RPC).

Designed NeonDB database schema: tables — users (firebase_uid, email, role), documents (student_id, doc_type, s3_url, sha256_hash, tx_hash, issued_at, status), audit_logs (user_id, action, ip_address, timestamp), notifications (doc_id, student_email, sns_message_id, ses_status, sent_at).

Executed NeonDB schema migration using Drizzle ORM scripts. Validated table creation via NeonDB console. Established foreign key relationships and UUID primary keys across all tables.

Created system architecture block diagram and Level-1 Data Flow Diagram for the mini-project report (Figures 6.1 and 6.4).

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Executed NeonDB schema migrations using Drizzle ORM. Validated all table structures and foreign key constraints in the NeonDB console. Wrote database helper functions for common query patterns.	Designed the full system architecture and drew block diagram and DFD diagrams. Configured AWS IAM policy with specific resource ARNs for S3, SNS, and SES. Documented architecture decisions.	N/A	N/A

Mentor's Suggestions:
Ensure database schema uses proper indexing on frequently-queried columns (student_id, sha256_hash). Confirm all IAM policy ARNs are correct and scoped to the exact resource names. Add audit_logs table triggers or application-level logging from day one.

Signature:
Team Member 1: Jason Gonsalves

Project guide:
Signature: _____

Team Member 2: Ashley Almeida Date: _____

WEEK – 03
Date: From 15 Jan 2026 To: 21 Jan 2026

Progress Achieved:
Developed Authblock Solidity smart contract (CredentialRegistry.sol) using Solidity 0.8.x. Contract maintains a mapping(string => DocumentRecord) where DocumentRecord stores bytes32 hash, address issuer, and uint256 timestamp.

Implemented two core functions: storeHash(string calldata docId, bytes32 hash) with onlyAdmin modifier for credential issuance, and verifyHash(string calldata docId, bytes32 hash) as a public view function returning (bool verified, uint256 timestamp).

Contract emits DocumentAnchored(string docId, bytes32 hash, uint256 timestamp) event on each storage call for off-chain indexing. Deployed successfully to Ethereum Sepolia Testnet using Hardhat. Contract address saved as NEXT_PUBLIC_CONTRACT_ADDRESS.

Wrote and executed Hardhat unit test suite: 100% pass rate covering storeHash, verifyHash, access control (non-admin rejection with revert), duplicate hash detection, and timestamp accuracy verification.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Wrote the complete CredentialRegistry.sol smart contract, Hardhat deployment script, and full unit test suite. Verified deployment on Etherscan Sepolia. Integrated ABI into Next.js project.	Studied Ethereum Solidity 0.8.x access control patterns (Ownable, custom onlyAdmin modifier). Tested all edge cases in Hardhat test suite. Reviewed contract ABI for ethers.js server-side integration.	N/A	N/A

Mentor's Suggestions:
Consider emitting a separate VerificationAttempted event in verifyHash for audit logging. Test the onlyAdmin revert with multiple unauthorized addresses. Ensure the contract ABI is versioned alongside the code in the repository.

Signature:
Team Member 1: Jason Gonsalves
Team Member 2: Ashley Almeida

Project guide:
Signature: _____
Date: _____

WEEK – 04
Date: From 22 Jan 2026 To: 28 Jan 2026

Progress Achieved:
Integrated Firebase Authentication into Next.js 14 App Router. Implemented email/password first-factor login and TOTP (Time-based One-Time Password) MFA second factor for the admin portal using Firebase Auth SDK v9 modular API.

Developed JWT verification middleware for all Next.js API routes. Middleware extracts and verifies Firebase ID tokens using Firebase Admin SDK, rejecting unauthenticated or expired tokens with HTTP 401 responses.

Implemented role-based routing middleware: admin routes redirect unauthenticated users to the admin login page; student routes redirect to student login; public verification page remains fully accessible without authentication.

Built Admin Login page UI with Tailwind CSS: email/password form with real-time validation, loading spinner, error messages, and a separate MFA TOTP input step that appears post-first-factor success.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Implemented Firebase Auth SDK v9 integration, JWT verification middleware for API route protection, and role-based routing middleware for admin/student/public access control.	Designed and built the Admin Login page UI with Tailwind CSS including form validation states, loading states, MFA TOTP input component, and responsive layout for all screen sizes.	N/A	N/A

Mentor's Suggestions:
Test MFA bypass scenarios: expired TOTP codes, replayed tokens, and concurrent session handling. Ensure JWT middleware short-circuits gracefully on malformed tokens without exposing stack traces. Proactively refresh Firebase tokens before expiry.

Signature:
Team Member 1: Jason Gonsalves
Team Member 2: Ashley Almeida

Project guide:
Signature: _____
Date: _____

WEEK – 05
Date: From 29 Jan 2026 To: 04 Feb 2026

Progress Achieved:
Developed the Admin Dashboard — primary interface for credential issuance. Features: paginated data table of all issued documents (with sort and filter), CSV upload form, manual student data entry form, and real-time issuance status indicators.

Integrated Papa Parse for client-side CSV parsing. CSV upload validates required fields (student_id, name, subject_marks) before submission, with clear error messages for malformed rows.

Implemented grade computation logic aligned with University of Mumbai norms: per-subject pass/fail based on 40% threshold, aggregate SGPA calculation, and final result determination.

Added recharts-based analytics panel showing credential issuance trends over configurable date ranges (weekly, monthly, semester-wide) with bar chart and line chart components.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Built the complete Admin Dashboard UI: paginated data table with sort/filter, CSV upload with Papa Parse validation, manual entry form, and real-time issuance status badges.	Implemented grade computation and SGPA calculation logic per Mumbai University norms. Built the recharts analytics panel with configurable date range selectors and chart type toggles.	N/A	N/A

Mentor's Suggestions:

Add a CSV template download button so admins know the exact expected column format. Add bulk re-issuance capability for batch corrections. Ensure pagination uses server-side querying to avoid loading all records into the browser.

Signature:
Team Member 1: Jason Gonsalves
Team Member 2: Ashley Almeida

Project guide:
Signature: _____
Date: _____

WEEK – 06

Date: From 05 Feb 2026 To: 11 Feb 2026

Progress Achieved:

Developed the PDF generation module. A hidden React component renders the marksheet template dynamically populated with student data (name, roll number, subject marks, grades, result, semester, academic year).

html2canvas converts the rendered DOM element to a canvas image; jspdf embeds this into a PDF document with institutional header, subject-wise marks table, result summary, and signature placeholder lines.

Implemented /api/generate-pdf API route that accepts student data JSON, generates the PDF buffer server-side, and returns it both for admin preview and direct use in S3 upload and SHA-256 hash computation.

Generated certificate PDF in addition to marksheet using @react-pdf/renderer for structured layout with institutional branding. Both PDFs are linked per document record in NeonDB.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4

Developed the complete PDF generation module using jspdf and html2canvas. Built the marksheet React template component with dynamic data binding and institutional header.	Built the certificate PDF template using @react-pdf/renderer. Integrated the PDF preview functionality in the Admin Dashboard as a confirmation step before final issuance.	N/A	N/A

Mentor's Suggestions:
Ensure PDF generation is fully deterministic — identical input data must always produce byte-identical PDF binary output to guarantee SHA-256 hash consistency across re-generations. Test with varied student data including special characters in names.

Signature:
Team Member 1: Jason Gonsalves
Team Member 2: Ashley Almeida

Project guide:
Signature: _____
Date: _____

WEEK – 07
Date: From 12 Feb 2026 To: 18 Feb 2026

Progress Achieved:
Implemented Amazon S3 integration using AWS SDK v3 PutObjectCommand. PDF buffers are uploaded to the designated S3 bucket (authblock-documents) with structured keys: documents/{year}/{studentId}/{docType}_{docId}.pdf.

Enabled S3 bucket versioning for accidental deletion recovery. Configured S3 lifecycle rules for cost management: transition to S3-IA after 90 days. Bucket blocks all public access; objects are accessible only via pre-signed URLs.

Generated server-side pre-signed URLs with 7-day expiry using GetObjectCommand for student download links. Verified storage integrity by downloading stored files and recomputing SHA-256 hashes to confirm match.

Computed SHA-256 hash of each PDF buffer using Node.js crypto module (crypto.createHash('sha256').update(pdfBuffer).digest('hex')) immediately after buffer generation. Hash stored alongside S3 URL in NeonDB documents table.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Implemented AWS S3 upload using SDK v3 PutObjectCommand, pre-signed URL generation for secure 7-day download links, and	Configured S3 bucket versioning, lifecycle rules, and public access block settings. Validated SHA-256 hash computation pipeline and	N/A	N/A

verified SHA-256 hash consistency between generated buffer and stored S3 object.	confirmed hash consistency in integration tests.		

Mentor's Suggestions:
Enable S3 server-side encryption (SSE-S3 or SSE-KMS) for all stored documents. Log all S3 API calls to CloudTrail. Add server-side validation that the uploaded file size matches the generated buffer size before confirming success.

Signature:
Team Member 1: Jason Gonsalves
Team Member 2: Ashley Almeida

Project guide:
Signature: _____
Date: _____

WEEK – 08
Date: From 19 Feb 2026 To: 25 Feb 2026

Progress Achieved:
Implemented the blockchain anchoring module. After successful S3 upload, the API route submits the SHA-256 hash to the smart contract storeHash() function via ethers.js provider configured with Alchemy Sepolia RPC and admin private key.

The resulting Ethereum transaction hash (txHash) is stored in NeonDB documents table, creating a permanent off-chain record of the on-chain anchoring event.

Implemented retry logic with exponential backoff for blockchain transaction submission to handle Sepolia network congestion. Transactions are monitored for confirmation with a 2-block confirmation threshold before marking issuance as complete.

Conducted end-to-end integration test of full pipeline (CSV upload → PDF generation → S3 storage → SHA-256 computation → Ethereum anchoring → NeonDB persistence) for 5 test student records. All steps completed successfully.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Implemented the blockchain anchoring module using ethers.js, including retry logic with exponential backoff and 2-block confirmation monitoring. Conducted end-to-end pipeline integration tests.	Monitored Sepolia Etherscan for transaction confirmations during testing. Documented the storeHash call flow and verified all NeonDB txHash records against Etherscan explorer for correctness.	N/A	N/A

Mentor's Suggestions:

Implement a background job to re-check pending transactions and update their confirmation status in NeonDB. Consider Merkle tree batch anchoring for gas optimization when issuing large numbers of credentials simultaneously.

Signature:
Team Member 1: Jason Gonsalves
Team Member 2: Ashley Almeida

Project guide:
Signature: _____
Date: _____

WEEK – 09
Date: From 26 Feb 2026 To: 04 Mar 2026

Progress Achieved:
Built the AWS event-driven notification pipeline. Upon successful blockchain anchoring, the API route publishes an issuance event to Amazon SNS topic (authblock-issuance-events) with payload: student_email, student_name, doc_id, marksheet_url, certificate_url, tx_hash, verification_url.

Configured Amazon SQS Standard Queue subscribed to the SNS topic. SQS acts as a buffer ensuring notification delivery is completely decoupled from the core issuance flow — email failures cannot block credential issuance.

Developed AWS Lambda function (authblock-notifier, Node.js 18.x runtime) triggered by SQS messages. Lambda uses AWS SES SDK v3 SendEmailCommand to send a formatted HTML email to the student with document links and verification page URL.

Set up SQS Dead-Letter Queue (DLQ) for failed Lambda invocations. Configured CloudWatch alarm on DLQ depth to alert admin when notification failures accumulate. Tested end-to-end: SNS publish → SQS → Lambda trigger → SES send — email delivered within 8 seconds.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Implemented the full SNS → SQS → Lambda notification pipeline. Wrote the Lambda function code in Node.js 18.x. Configured SQS DLQ for failed messages and CloudWatch alarm on DLQ depth.	Designed the HTML email template for student credential notifications including marksheet/certificate download links, blockchain transaction URL, and verification page link. Tested SES bounce handling.	N/A	N/A

Mentor's Suggestions:
Test the SQS buffering behavior explicitly: throttle SES and verify that messages are buffered in SQS without data loss, then resume and confirm all messages are processed in order. Monitor SES bounce and complaint rates from day one to protect sender reputation.

Signature: **Project guide:**

Team Member 1: Jason Gonsalves Signature: _____
Team Member 2: Ashley Almeida Date: _____

WEEK – 10
Date: From 05 Mar 2026 To: 11 Mar 2026

Progress Achieved:
Developed the Student Portal — a Firebase-authenticated interface for students to view their credentials. Portal fetches student-specific documents from the /api/student/documents endpoint, displaying credential cards with subject marks, result, SGPA, and grade.

Each credential card shows: marksheet download link (pre-signed S3 URL), certificate download link, live blockchain verification status badge (green Verified / red Tampered) queried in real-time from the smart contract via ethers.js.

Built the Public Verification Page — fully accessible without authentication. Accepts a Document ID input, queries NeonDB for stored SHA-256 hash and S3 URL, calls smart contract verifyHash() view function, and displays Verified (with original issuance timestamp) or Tampered / Not Found status.

Added QR code generation for each credential card encoding the verification URL with Document ID, enabling easy third-party scanning using the qrcode.react library.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Developed the Student Portal UI with credential cards, live blockchain verification status badges, download links, QR code generation, and Firebase student authentication flow.	Built the Public Verification Page with Document ID lookup, smart contract live query, tamper-detection status display, and issuance timestamp rendering. Added rate limiting middleware to prevent abuse.	N/A	N/A

Mentor's Suggestions:
Ensure pre-signed S3 URLs refresh on each student portal load to prevent expiry issues. Add server-side rate limiting to the public verification endpoint using a token-bucket algorithm to prevent automated scraping.

Signature: **Project guide:**
Team Member 1: Jason Gonsalves Signature: _____
Team Member 2: Ashley Almeida Date: _____

WEEK – 11
Date: From 12 Mar 2026 To: 18 Mar 2026

Progress Achieved:
Configured AWS CloudWatch alarms: Lambda error rate (threshold >2 errors/5 min), SQS queue depth (threshold >100 messages), S3 API error rate, and NeonDB connection pool exhaustion alert.

Enabled AWS CloudTrail for complete API activity auditing. CloudTrail logs stored in dedicated S3 bucket (authblock-audit-logs) with 90-day retention and SSE-KMS encryption.

Implemented SES bounce and complaint SNS notification handling with automatic suppression list management to protect sender reputation. Monitored SES reputation dashboard — bounce rate <0.1%, complaint rate <0.01%.

Performed security testing: IAM policy boundary validation (confirmed AccessDenied for all out-of-scope AWS resources), Firebase MFA bypass attempts (all blocked), SQL injection testing on all NeonDB query parameters (all parameterized — no injection possible), and XSS testing on all user-input API fields.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Configured CloudWatch alarms for Lambda, SQS, S3, and NeonDB. Enabled CloudTrail with KMS-encrypted audit log storage. Performed IAM policy boundary validation and SQL injection security tests.	Implemented SES bounce/complaint SNS handling and suppression list management. Conducted Firebase MFA bypass and XSS security tests. Documented all security test results with pass/fail status.	N/A	N/A

Mentor's Suggestions:
Ensure CloudTrail logs are not stored in the same S3 bucket as the credential documents. Schedule a monthly SES reputation review. Perform a final penetration test covering all public API endpoints before the project demonstration.

Signature:
Team Member 1: Jason Gonsalves
Team Member 2: Ashley Almeida

Project guide:
Signature: _____
Date: _____

WEEK – 12
Date: From 19 Mar 2026 To: 25 Mar 2026

Progress Achieved:
Conducted full-scale functional testing: batch of 50 student records uploaded via CSV. System generated 50 PDF marksheets and 50 certificates, anchored all 100 SHA-256 hashes to Ethereum Sepolia blockchain, and dispatched 50 notification emails via SES. Average end-to-end issuance time: 3.2 seconds per credential.

Tampering test: 20 PDFs modified using binary hex editor and re-uploaded to S3. Public Verification Page correctly identified all 20 as Tampered (100% detection accuracy, n=20). All unmodified documents verified successfully (100% accuracy, n=80).

Email delivery metrics: 100% delivery rate, 0 bounces, 0 spam complaints in full test run. AWS SES bounce rate 0% and complaint rate 0% — well within recommended thresholds (<5% bounce, <0.1% complaint).

Completed and submitted all chapters of the mini-project report (Ch. 1-8, References, Appendix I & II) following the college format. Prepared 10-minute project demonstration covering admin issuance, Etherscan confirmation, student email, and public tamper-detection verification.

Team Member's contribution:

Team Member 1	Team Member 2	Team Member 3	Team Member 4
Conducted batch issuance testing (50 records), tampering detection tests (20 modified PDFs), and performance benchmarking. Completed all implementation modules and finalized the project codebase.	Compiled and formatted the full mini-project report: all chapters, literature review table, results analysis, IEEE-format references, and Appendices. Prepared project demonstration script and slide deck.	N/A	N/A

Mentor's Suggestions:

Prepare a clear and concise 10-minute demonstration flow: (1) admin CSV upload and issuance, (2) Etherscan Sepolia transaction confirmation, (3) student email receipt and portal view, (4) public verification with a legitimate document, (5) public verification with a tampered document showing detection. Finalize the logbook.

Signature:

Team Member 1: Jason Gonsalves

Team Member 2: Ashley Almeida

Project guide:

Signature: _____

Date: _____

Details of Publication/Patents/Participation/Awards

Technical Paper: A technical paper titled 'Authblock: Blockchain-Based Academic Credential Verification Using Ethereum and AWS Cloud Infrastructure' has been drafted and is being reviewed for submission to a national-level technical conference. The paper covers the system architecture, SHA-256 cryptographic hashing approach, Solidity smart contract design, and the AWS event-driven notification pipeline (SNS → SQS → Lambda → SES).

Participation: The Authblock project was presented at the C.R.C.E. Internal Project Showcase (Semester VI, AY 2025-2026). The team received commendation for the end-to-end integration of Ethereum blockchain and AWS cloud infrastructure in an academically relevant use case addressing credential fraud prevention.

Self-Learning: Both team members completed the AWS Cloud Practitioner Essentials online training during the project period to strengthen practical understanding of S3, SNS, SQS, Lambda, SES, CloudWatch, and CloudTrail services integrated into the Authblock system.

Patents: No patent filed for the current semester. The team explored the novelty of the Authblock end-to-end credential pipeline (PDF generation → S3 storage → SHA-256 hashing → Ethereum anchoring → event-driven notification) for potential provisional patent filing in a future semester.